SCALE FOR PROJECT PYTHON MODULE 06 (/PROJECTS/PYTHON-MODULE-06)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere-v2.42madrid.com:vogsphere/intra-uuid-3c719b71-b39f-479
```

Introduction

- Remain polite, courteous, respectful, and constructive throughout the evaluation process. The well-being of the community depends on it.

- Together with the person (or group) being evaluated, identify any issues in the work. Take the time to discuss and debate the problems you have identified.

- You must consider that there might be differences in how your peers have understood the project's instructions and the scope of its functionality. Always keep an open mind and grade their work as honestly as possible. The pedagogy is valid only if peer evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student's or group's Git repository.

- Double-check that the Git repository belongs to the student or the group. Ensure that the work is for the relevant project and also check that the "git clone" command is run in an empty folder.

- Check carefully that no malicious aliases were used to fool you and make you evaluate something other than the content of the official repository.

- To avoid any surprises, carefully check that both the evaluating and the evaluated students have reviewed any scripts that may be used to facilitate grading.

- If the evaluating student has not completed that particular project yet, it is mandatory for this student to read the entire subject document prior to starting the defense.

- Use the flags available on this scale to signal an empty repository, a non-functioning program, a norm error, cheating, etc. In these cases, the grading is over and the final grade is 0 (or -42 in the case of cheating). However, with the exception of cheating, you are encouraged to continue to discuss your work (even if you have not finished it) in order to identify any issues that may have caused this failure and to avoid repeating the same mistakes in the future.

- Remember that for the duration of the defense, no unexpected, premature, uncontrolled termination of the program is allowed (except if exceptionally allowed);

otherwise, the final grade is 0. Use the appropriate flag.
You should never have to edit any file other than the configuration file, if one exists.
If you want to edit a file, take the time to discuss the reasons with the
evaluated student and make sure both of you agree with this change.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/203229/en.subject.pdf)

Preliminaries

Basics

- Only grade the work that is in the learner's Git repository.
- Ensure the Git repository belongs to the learner.
- Check that no malicious aliases are used.
- Verify that the project structure matches the subject requirements and the file tree.

Are all preliminary checks satisfied?

Yes No

Part I - The Alembic

Alembic experiment

Test the Alembic scripts:

- Run: python3 ft_alembic_[0-5].py
- The output of each script should demonstrate the same elements as in the example
- ft_alembic_0 uses "import elements" and calls "elements.create_fire()"
- ft_alembic_1 uses "from elements import create_water" and calls "create_water()"
- ft_alembic_2 uses "import alchemy.elements" and calls "alchemy.elements.create_earth()"
- ft_alembic_3 uses "from alchemy.elements import create_air" and calls "create_air()"
- ft_alembic_4 uses "import alchemy" and calls "alchemy.create_air()". It then tries to access "alchemy.create_earth()", and it should fail (uncaught exception allowed).
- ft_alembic_5 uses "from alchemy import create_air" and calls "create_air()"
- Verify that the alchemy package has an **init**.py file in the alchemy folder, and that it imports "create_air" from ".elements" (alternative solutions exist and are accepted) but not "create_earth". Other elements in this file will be controlled in the upcoming questions.

Yes No

Part II - Distillation

Import Distillation experiment

Test the Distillation experiment:

- Run: python3 ft_distillation_[0-1].py
- Verify that direct access to the potions.py file is used and works in distillation_0, testing the strength and healing potions.
- Verify that the direct use of the alchemy module works in distillation_1, testing the strength potion.
- Verify that the healing potion is called using the "heal()" alias in distillation_1.
- Verify that the **init**.py file for the alchemy module has been updated to expose the potions.
- Check that potions.py correctly imports elements.py and alchemy/elements.py.
- Confirm that the learner understands the differences between import methods.

Yes No

Part III - The Great Transmutation

Transmutation experiment

Test the Transmutation experiment:

- Run: python3 ft_transmutation_[0-2].py
- Verify access to the transmutation recipe "lead_to_gold" using:
  - the alchemy/transmutation/recipes.py file directly
  - the transmutation module
  - the alchemy module
- Check that the transmutation module has an **init**.py file.
- Check that the **init**.py file for the alchemy module has been updated to expose "lead_to_gold".
- Check that the recipes.py file contains at least one absolute import and one relative import.
- Verify that the learner understands when to use each approach.

Yes No

Part IV - Avoid the Explosion

Circular experiment

Test the Circular experiment:

- Run: python3 ft_kaboom_[0-1].py
- Verify that both scripts show a working and a failing import (uncaught exception allowed).
- Check that the alchemy/grimoire module contains the expected files: the light_* and dark_* Python files, and the **init** file.
- Check that ingredients are defined in the spellbook and not in the validator, and that both files need each others.
- Check that "Earth, Wind & Fire" can record the spell "Fantasy"
- Verify that the circular dependency problem is explained
- Ask the learner which solution was used to avoid circular dependencies.

Yes No

Code Quality

Import Mastery Assessment

Evaluate the overall code quality:

- Is the code written in Python 3.10 or higher?
- Does the code adhere to the flake8 linter standards (no errors)?
- Comprehensive type annotations are used throughout (all functions have parameter and return types). Check this using mypy (note that there is an exception for ft_alembic_4.py).
- Docstrings are NOT required for this module.
- Is the overall code organization clean and professional?

Yes No

Ratings

Don’t forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work Incomplete work Invalid compilation Norme Cheat Crash

▲ Concerning situation Leaks Forbidden function Can't support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33) Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2) Privacy policy (https://profile.intra.42.fr/legal/terms/5) General term of use of the site (https://profile.intra.42.fr/legal/terms/6) Rules of procedure (https://profile.intra.42.fr/legal/terms/4) Terms of use for video surveillance (https://profile.intra.42.fr/legal/terms/1) Legal notices (https://profile.intra.42.fr/legal/terms/3)